

Yale University

Multilayer Perceptrons, Backpropagation, and Training

S&DS 265 · Introductory Machine Learning · Lecture 11

Zhuoran Yang

Fall 2026

AI usage: figures were generated, and these slides polished, with the help of AI tools.

Outline

1. Why Deep Learning Now?
2. Depth and Learned Representations
3. Backpropagation and Automatic Differentiation
4. Stabilizing Deep Networks
5. Regularization and the Training Handoff

Learning Objectives

In this lecture you will learn:

1. Why deep learning emerged from the joint growth of architectures, data, computation, and trainability;
2. How depth, width, and nonlinear activations define the MLP model class and its cost;
3. How a scalar chain rule becomes a clear layer-by-layer backpropagation algorithm;
4. How initialization and residual pathways keep signals and gradients usable across depth;
5. Why dropout targets overfitting rather than unstable optimization.

Outline

1. Why Deep Learning Now?
2. Depth and Learned Representations
3. Backpropagation and Automatic Differentiation
4. Stabilizing Deep Networks
5. Regularization and the Training Handoff

Motivating Example: Recognizing Handwritten Digits

An image has $28 \times 28 = 784$ pixel values, while the target is one of ten digits. A linear classifier can weight pixels, but it cannot build a hierarchy of strokes, loops, and shapes.

How can a model learn the intermediate features it needs?

Example data: MNIST handwritten digits, LeCun et al. (1998).

The MNIST Data



Source: MNIST examples; LeCun, Cortes, and Burges.

Observed input

$x_i \in [0, 1]^{784}$: a flattened 28×28 grayscale image.

Target

$y_i \in \{0, \dots, 9\}$: the digit identity.

Standard MNIST contains 60,000 training images and 10,000 test images.

Reading One Image



A 10x10 grid of handwritten digits from 0 to 9, representing a vector of intensities. The digits are arranged in rows: Row 1: 0 0 0 0 0 0 0 0 0 0; Row 2: 1 1 1 1 1 1 1 1 1 1; Row 3: 2 2 2 2 2 2 2 2 2 2; Row 4: 3 3 3 3 3 3 3 3 3 3; Row 5: 4 4 4 4 4 4 4 4 4 4; Row 6: 5 5 5 5 5 5 5 5 5 5; Row 7: 6 6 6 6 6 6 6 6 6 6; Row 8: 7 7 7 7 7 7 7 7 7 7; Row 9: 8 8 8 8 8 8 8 8 8 8; Row 10: 9 9 9 9 9 9 9 9 9 9.

One observation is not a symbol already understood by the model. It is a vector of intensities

$$x_i = (x_{i1}, \dots, x_{i,784}) \in [0, 1]^{784}.$$

The label supplies only the final answer y_i ; it does not identify strokes, endpoints, loops, or any other useful intermediate feature.

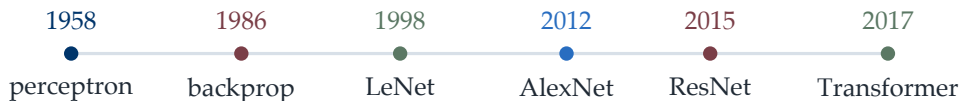
Problem Setup

Given $\mathcal{D}_n = \{(x_i, y_i)\}_{i=1}^n$, the population target is the conditional distribution $p^*(y | x)$. We restrict the estimator to an architecture-dependent MLP class $\mathcal{F}_{\text{MLP}}$ and fit

$$\hat{\theta} \in \arg \min_{\theta} \hat{R}_n(\theta), \quad \hat{R}_n(\theta) = -\frac{1}{n} \sum_{i=1}^n \log p_{\theta}(y_i | x_i).$$

Earlier models fixed the features. An MLP **learns features and classifier jointly**. Its **price** is many parameters, nonconvex optimization, and representations that are harder to inspect. Held-out cross-entropy and error remain the evidence.

Neural Networks Have a Long History



- ▶ The basic neuron and layered computation predate modern software;
- ▶ Backpropagation made hidden representations trainable by gradient descent;
- ▶ Later breakthroughs changed architectures and the scale of data and computation.

Source: Rosenblatt (1958); Rumelhart, Hinton, and Williams (1986); LeCun et al. (1998); Krizhevsky et al. (2012); He et al. (2016); Vaswani et al. (2017).

1958: The Perceptron Learns a Linear Decision Rule

Psychological Review

THE PERCEPTION: A PROBABILISTIC MODEL FOR INFORMATION STORAGE AND ORGANIZATION IN THE BRAIN¹

P. ROSENBLATT

Cornell Aeronautical Laboratory

If we are eventually to understand the capability of higher organisms for perceptual recognition, generalization, recall, and thinking, we must first have answers to three fundamental questions:

1. How is information about the physical world sensed, or detected, by the biological system?

2. In what form is information stored, or remembered?

3. How does information contained in storage, or in memory, influence recognition and behavior?

The first of these questions is in the province of sensory physiology, and is the only one for which appreciable understanding has been attained. This article will be concerned primarily with the second and third questions, which are still subject to vast amounts of speculation, and where the few relevant facts currently supplied by neurophysiology have not yet been integrated into an acceptable theory.

With regard to the second question, two alternative positions have been maintained. The first suggests that storage of sensory information is in the form of coded representations or images, with some sort of one-to-one mapping between the sensory stimulus

and the stored pattern. According to this hypothesis, if one understood the code or "writing diagram" of the sensory system, one should, in principle, be able to discover exactly what an organism remembers by reconstructing the original sensory patterns from the "memory traces" which they have left, much as we might develop a photographic negative, or translate the pattern of electrical changes in the "memory" of a digital computer. This hypothesis is appealing in its simplicity and easily intelligibility, and a large family of theoretical brain models has been developed around the idea of a coded, representational memory (2, 3, 8, 9, 14). The alternative approach, which stems from the tradition of British empiricism, holds that the same that the images of stimuli may never really be recorded at all, and that the "control" system organically acts as an intricate switching network, where retention takes the form of some connection, or pathways, between centers of activity. In many of the more recent developments of this position (Hull's "cell assemblies," and Hull's "cortical anticipatory goal response," for example the "responses" which are imprinted to stimuli and which are organized within the CNS itself. In this case the response represents an "idea" rather than an action. The important feature of this approach is that there is never any simple mapping of the stimulus into memory, according to some code which would permit its later reconstruction. Whatever is

The perceptron adapts a weighted sum of inputs:

$$s(x) = w^T x + b, \quad \hat{y} = \mathbf{1}\{s(x) > 0\}.$$

This is an important ancestor of modern networks, but one such unit still produces a **linear boundary**. Useful intermediate features must either be supplied by hand or learned by additional layers.

Source: First page of Rosenblatt, Psychological Review 65(6), 1958; DOI 10.1037/h0042519.

10/61

11 / 61

12/61

Modern Success Required Four Ingredients

architecture
layers and
inductive
bias

data
large labeled
or self-
supervised
corpora

compute
GPUs and
scalable
kernels

trainability
backprop,
initialization,
optimization

The mathematics of the chain rule was not new in 2012. What changed was the ability to fit large compositions reliably on rich datasets with useful architectural structure.

Deep learning is architecture, computation, and scale working together.

Outline

1. Why Deep Learning Now?
2. Depth and Learned Representations
3. Backpropagation and Automatic Differentiation
4. Stabilizing Deep Networks
5. Regularization and the Training Handoff

From One Hidden Layer to Several

The depth-one regression network from Lecture 6 was

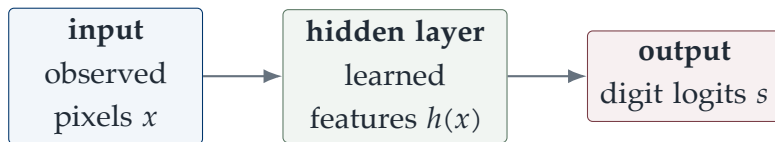
$$h(x) = \sigma(W_1x + b_1), \quad f_{\theta}(x) = a^{\top}h(x).$$

One hidden layer already turns learned features into a prediction. For digit recognition, we extend the same construction in two ways:

1. Repeat the hidden transformation several times;
2. Use ten output logits and a softmax cross-entropy loss.

Depth lets later features reuse earlier ones. The new question is how to differentiate and train the longer composition.

A Hidden Layer Is an Intermediate Representation



The input and label are observed. Hidden coordinates are not labeled as “stroke” or “loop.” Their meaning is determined indirectly by whichever values help reduce the final prediction loss.

Hidden means internal to the model, not mysterious or unobservable to us.

An MLP Is a Composition of Layers

Let $h^{(0)} = x$. For hidden layers $\ell = 1, \dots, L$,

$$z^{(\ell)} = W^{(\ell)}h^{(\ell-1)} + b^{(\ell)}, \quad h^{(\ell)} = \sigma_{\ell}(z^{(\ell)}).$$

The output layer produces logits and probabilities:

$$s = W^{(L+1)}h^{(L)} + b^{(L+1)}, \quad p_k = \frac{e^{s_k}}{\sum_{j=1}^{10} e^{s_j}}.$$

Every coordinate in one layer connects to every coordinate in the next, hence **fully connected**.

Activations Keep the Composition Nonlinear

Lecture 6 introduced the main choices:

$$\text{ReLU}(z) = \max(0, z), \quad \text{GELU}(z) = z\Phi(z), \quad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

- ▶ ReLU is cheap and piecewise linear.
- ▶ GELU is a smooth gate used in many modern transformer MLP blocks.
- ▶ Sigmoid is useful for probabilities, but its flat tails can attenuate hidden-layer gradients.

Without an activation, several affine layers collapse to one affine map; the nonlinear gate is what enlarges the function class.

The MNIST MLP Has Explicit Tensor Shapes

For a minibatch of B images and widths $784 \rightarrow 256 \rightarrow 128 \rightarrow 10$,

$$X : [B, 784] \longrightarrow H^{(1)} : [B, 256] \longrightarrow H^{(2)} : [B, 128] \longrightarrow S : [B, 10].$$

The parameters have shapes

$$W^{(1)} : [256, 784], \quad W^{(2)} : [128, 256], \quad W^{(3)} : [10, 128],$$

plus one bias per output coordinate.

Writing shapes before code catches many broadcasting and orientation errors.

A Small MLP Already Has Many Parameters

The parameter count is

$$(784 \cdot 256 + 256) + (256 \cdot 128 + 128) + (128 \cdot 10 + 10) \\ = \boxed{235,146}.$$

That is modest by modern standards, but already enough that writing and updating each derivative separately is impractical.

Architecture determines the function class and the cost of parameters, activations, gradients, and optimizer state.

Width and Depth Enlarge the Function Class Differently

Width

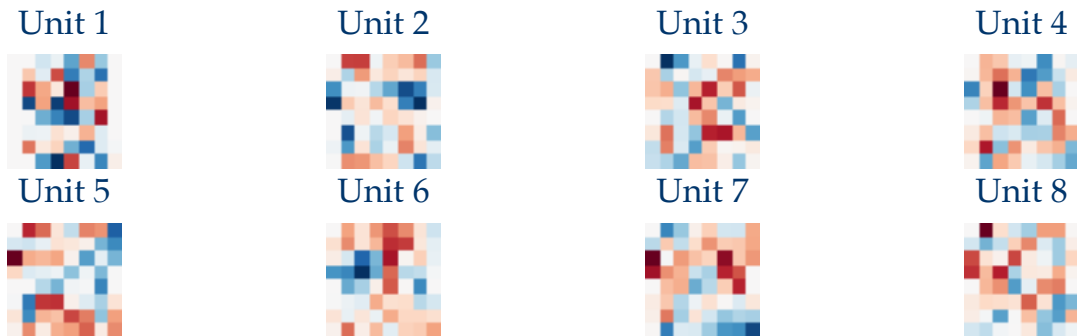
Adds parallel features within a layer. A sufficiently wide one-hidden-layer network can approximate many continuous functions on compact sets.

Depth

Composes features. Later units can reuse patterns built by earlier units, often representing compositional structure more efficiently.

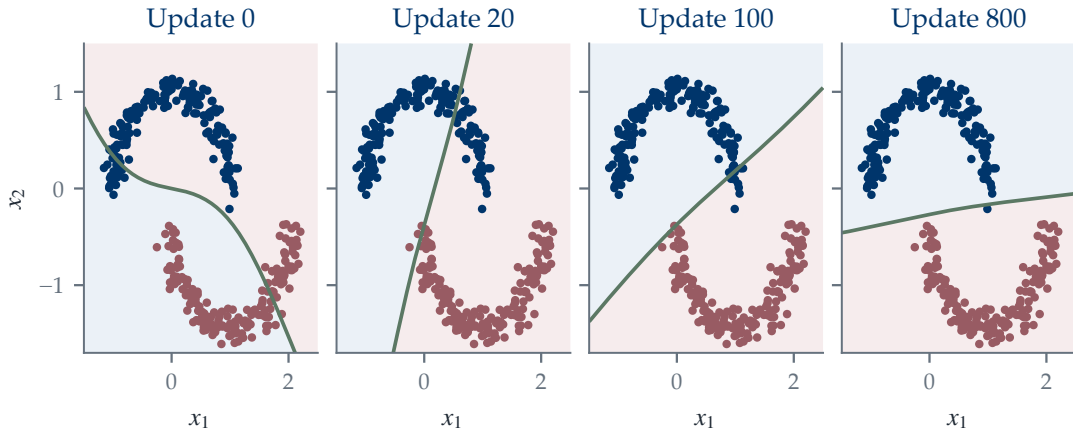
Universal approximation is an existence result; it does not say that finite data and a chosen optimizer will find the useful function.

Hidden Features Become Class-Specific Templates



Course calculation from a fitted MLP on handwritten digits.

The Decision Rule Changes Throughout Training



Course from-scratch MLP on a declared two-dimensional classification task.

Classroom Demo: Watch an MLP Learn

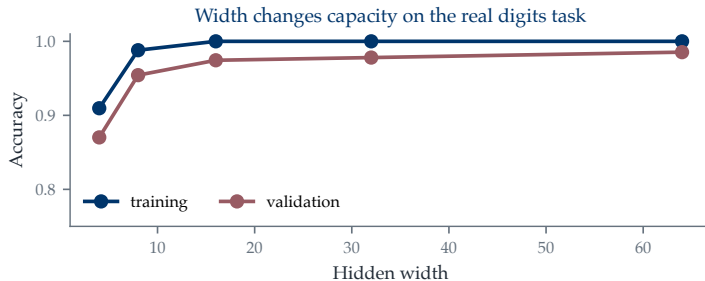
Change one quantity at a time in a two-dimensional classification problem:

1. Compare a narrow and a wider hidden layer;
2. Advance optimization updates and watch the boundary move;
3. Increase the step size until training becomes unstable;
4. Reset the random seed and compare the learned representation.

Question for the class

Which changes alter the **function class**, and which alter only the **training path** within that class?

Width and Validation Accuracy



Course fixed-split experiment on handwritten digits.

A Forward Pass Must Retain Intermediate Values

To compute predictions, the model forms

$$X, Z^{(1)}, H^{(1)}, \dots, Z^{(L)}, H^{(L)}, S, L.$$

Backward rules need selected forward values such as $H^{(\ell-1)}$, $Z^{(\ell)}$, and the logits or probabilities.

Training-memory consequence

Parameters are not the only large objects. Activations scale with minibatch size, sequence or image size, width, and depth; they are normally retained until their gradients have been computed.

Outline

1. Why Deep Learning Now?
2. Depth and Learned Representations
3. Backpropagation and Automatic Differentiation
4. Stabilizing Deep Networks
5. Regularization and the Training Handoff

Training Requires One Gradient per Parameter

The loss is one number, but the MLP has many parameters:

$$L(\theta), \quad \theta = \{W^{(1)}, b^{(1)}, \dots, W^{(L+1)}, b^{(L+1)}\}.$$

Gradient descent needs

$$\nabla_{\theta} L = (\nabla_{W^{(1)}} L, \nabla_{b^{(1)}} L, \dots, \nabla_{W^{(L+1)}} L, \nabla_{b^{(L+1)}} L).$$

Computing each derivative independently would repeat most of the same work.

Backpropagation reuses intermediate calculations while applying the chain rule from the loss back to every parameter.

Begin with the Scalar Chain Rule

Suppose

$$u = f(v), \quad L = g(u).$$

Then

$$\frac{dL}{dv} = \frac{dL}{du} \frac{du}{dv}.$$

Read the two colored factors as

$$\underbrace{\text{how the loss changes with } u}_{\text{arrives from downstream}} \times \underbrace{\text{how } u \text{ changes with } v}_{\text{local rule}}.$$

Backpropagation repeatedly performs this multiplication in reverse computational order.

A Short Notation for Loss Gradients

For any intermediate value v , define

$$\boxed{\bar{v} := \nabla_v L.}$$

The overline always means **gradient of the final loss with respect to this quantity**.
It has the same shape as v .

The scalar chain rule becomes

$$\bar{v} = \bar{u} \frac{du}{dv}.$$

incoming gradient $\times$ local derivative $=$ gradient passed backward

Worked Scalar Neuron: Forward Pass

Consider one ReLU unit with a squared-error loss:

$$z = wx + b, \quad h = \max(0, z), \quad L = \frac{1}{2}(h - y)^2.$$

Use

$$x = 2, \quad w = -1, \quad b = 3, \quad y = 4.$$

The forward values are

$$z = (-1)(2) + 3 = 1, \quad h = 1, \quad L = \frac{1}{2}(1 - 4)^2 = 4.5.$$

These saved values are exactly what the local backward rules will use.

Worked Scalar Neuron: Backward Pass

Start at the loss and move backward:

$$\bar{h} = \frac{\partial L}{\partial h} = h - y = -3,$$

$$\bar{z} = \bar{h} \mathbf{1}\{z > 0\} = -3,$$

$$\bar{w} = \bar{z} \frac{\partial z}{\partial w} = (-3)x = -6,$$

$$\bar{b} = \bar{z} \frac{\partial z}{\partial b} = -3.$$

Gradient descent increases w because $w \leftarrow w - \eta \bar{w}$ and $\bar{w} < 0$, moving the prediction toward $y = 4$.

One Vector Layer Has Two Local Operations

For an input vector $h \in \mathbb{R}^d$ and m output units,

$$\underbrace{z = Wh + b}_{\text{affine map}}, \quad \underbrace{u = \sigma(z)}_{\text{elementwise activation}},$$

with

$$W \in \mathbb{R}^{m \times d}, \quad b, z, u \in \mathbb{R}^m.$$

The backward pass receives $\bar{u} = \nabla_u L$ and must compute

$$\bar{z}, \quad \bar{W}, \quad \bar{b}, \quad \bar{h}.$$

We handle the activation and affine map separately.

Backward through an Elementwise Activation

Each output coordinate satisfies $u_j = \sigma(z_j)$, so

$$\frac{\partial L}{\partial z_j} = \frac{\partial L}{\partial u_j} \sigma'(z_j).$$

Stacking all coordinates gives

$$\bar{z} = \bar{u} \odot \sigma'(z).$$

The Hadamard product $\odot$ multiplies coordinate by coordinate.

For ReLU, $\sigma'(z_j)$ is 1 when $z_j > 0$ and 0 when $z_j < 0$; the gate either passes or blocks that coordinate's gradient.

Backward through an Affine Map

For $z = Wh + b$, the incoming gradient is $\bar{z}$. The local rules are

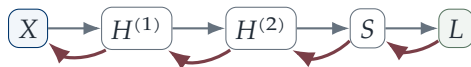
$$\boxed{\bar{W} = \bar{z}h^\top}, \quad \boxed{\bar{b} = \bar{z}}, \quad \boxed{\bar{h} = W^\top \bar{z}}.$$

Their shapes verify the formulas:

$$[m \times d] = [m \times 1][1 \times d], \quad [d \times 1] = [d \times m][m \times 1].$$

The outer product $\bar{z}h^\top$ assigns a gradient to every connection between the input and output coordinates.

Backpropagation Traverses the Graph in Reverse



The forward pass computes and saves intermediate values. The backward pass begins with the scalar loss and applies each local rule in reverse order. Contributions add when several downstream paths depend on the same value.

A One-Hidden-Layer Classifier Backpropagates in Order

For one example,

$$z^{(1)} = W^{(1)}x + b^{(1)}, \quad h^{(1)} = \sigma(z^{(1)}), \quad s = W^{(2)}h^{(1)} + b^{(2)}.$$

Softmax cross-entropy starts the reverse pass with

$$\bar{s} = p - e_y.$$

Then compute, in this order,

$$\bar{W}^{(2)} = \bar{s} h^{(1)\top},$$

$$\bar{h}^{(1)} = W^{(2)\top} \bar{s},$$

$$\bar{z}^{(1)} = \bar{h}^{(1)} \odot \sigma'(z^{(1)}),$$

$$\bar{W}^{(1)} = \bar{z}^{(1)} x^\top.$$

The Same Gradient Pattern Appears in Every Model

model	error signal	times feature
least squares (L04)	$X\theta - y$	x_i
logistic (L09)	$p_i - y_i$	x_i
softmax (L09)	$p_i - e_{y_i}$	x_i
MLP layer ℓ	$\bar{z}^{(\ell)}$	$h^{(\ell-1)}$

At the output, labels provide the error signal $p - e_y$. A hidden layer has no target label; its error signal is produced by the chain rule from the layers that follow it.

A Minibatch Produces Matrix Gradients

Store observations as rows:

$$Z = HW^{\top} + \mathbf{1}b^{\top}, \quad H \in \mathbb{R}^{B \times d}, \quad Z, \bar{Z} \in \mathbb{R}^{B \times m}.$$

Summing the B example contributions gives

$$\boxed{\bar{W} = \bar{Z}^{\top} H}, \quad \boxed{\bar{b} = \bar{Z}^{\top} \mathbf{1}}, \quad \boxed{\bar{H} = \bar{Z} W}.$$

If the loss averages the minibatch, its factor $1/B$ is already contained in $\bar{Z}$.

Automatic Differentiation Executes These Local Rules

During the forward pass, PyTorch records the operations actually executed and the values required by their backward rules. Calling

```
loss.backward()
```

traverses that graph in reverse and accumulates every leaf parameter's gradient in its `.grad` field.

Autodiff removes **derivative bookkeeping**; it **does not choose** the loss, initialize the model, select the optimizer, or guarantee a correct program.

Source: The demo notebook checks the hand-derived gradients against `loss.backward()` and against finite differences on a small deterministic problem.

One PyTorch Training Step

```
optimizer.zero_grad()
logits = model(x_batch)      # forward; save activations
loss = loss_fn(logits, y_batch)
loss.backward()              # reverse mode; fill .grad
optimizer.step()             # update parameters and state
```

The four operations have different roles. In particular, `backward()` computes gradients but does not update the model.

Outline

1. Why Deep Learning Now?
2. Depth and Learned Representations
3. Backpropagation and Automatic Differentiation
4. Stabilizing Deep Networks
5. Regularization and the Training Handoff

One Problem, Several Remedies

Backpropagation multiplies the gradient by one factor per layer, so depth **compounds** whatever that factor is. Too small and the early layers stop learning; too large and the updates blow up.

One Problem, Several Remedies

Backpropagation multiplies the gradient by one factor per layer, so depth **compounds** whatever that factor is. Too small and the early layers stop learning; too large and the updates blow up.

Guiding principle: keep that factor near **one**, forward and backward alike.

One Problem, Several Remedies

Backpropagation multiplies the gradient by one factor per layer, so depth **compounds** whatever that factor is. Too small and the early layers stop learning; too large and the updates blow up.

Guiding principle: keep that factor near **one**, forward and backward alike.

- ▶ **Initialization** sets the factor to about one **at the start**, through the scale of the random weights.
- ▶ **Skip connections** keep a factor of exactly one available **at all times**, through an identity path.
- ▶ **Clipping** and **schedules** bound the step when the factor is wrong anyway.

Zero Initialization Makes Hidden Units Identical

If every row of W_1 and every entry of b_1 begins equal, then every hidden unit produces the same value and receives the same gradient.

Gradient descent preserves this symmetry, so width does not create distinct learned features.

First requirement

Random initialization breaks symmetry among hidden units.

Arbitrary Random Scale Can Destroy the Signal

Suppose $z_j = \sum_{k=1}^{n_{\text{in}}} W_{jk} h_k$, with independent centered terms. Approximately,

$$\text{Var}(z_j) \approx n_{\text{in}} \text{Var}(W_{jk}) \text{Var}(h_k).$$

Repeating the layer can make activations and gradients vanish or explode if this scale changes systematically with depth.

Initialization is an architectural calculation, not just a random seed.

Xavier Balances Forward and Backward Scale

Glorot–Xavier initialization uses

$$\text{Var}(W_{jk}) = \frac{2}{n_{\text{in}} + n_{\text{out}}}.$$

It compromises between preserving the forward activation variance ($1/n_{\text{in}}$) and the backward gradient variance ($1/n_{\text{out}}$).

This argument assumes centered, weakly dependent signals and is best matched to tanh-like activations.

Source: Glorot and Bengio, 2010.

He Initialization Accounts for ReLU Gating

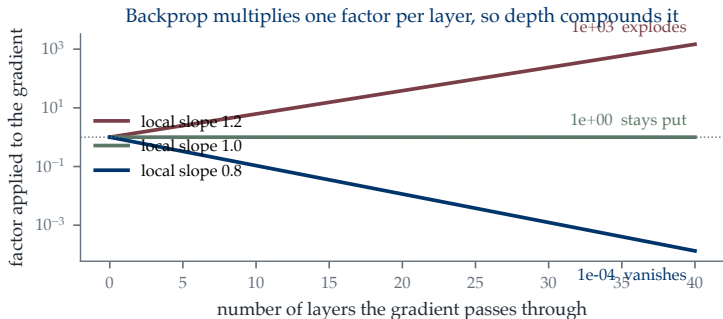
A centered symmetric input passes through a ReLU gate only about half the time, motivating

$$\text{Var}(W_{jk}) \approx \frac{2}{n_{\text{in}}}.$$

PyTorch layers expose initialization explicitly; the appropriate rule depends on the activation and parameterization.

A good initialization places training in a usable scale regime. It does not eliminate the need to monitor activations and gradient norms.

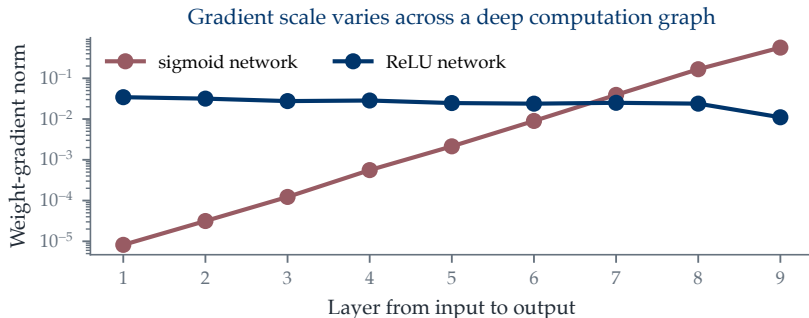
Depth Multiplies Local Gradient Scales



Backpropagation multiplies the gradient by one local slope per layer, so after L layers the factor is $(\text{slope})^L$. A slope of 0.8 leaves 10^{-4} of the gradient after 40 layers, 1.2 leaves 10^3 times too much, and only 1.0 transmits it unchanged.

Constant local slopes, so the product is exact.

Initialization and Activation Change Layerwise Gradients



The same nine-layer network on one digits minibatch, once with sigmoid and once with ReLU. The sigmoid gradient shrinks by **four orders of magnitude** between the output layer and the input layer, so the early layers barely move; with ReLU the norm is **about the same in every layer**.

Course calculation on a fixed handwritten-digits minibatch.

A Residual Block Learns a Correction

A plain block must learn the whole mapping, $h^{(\ell+1)} = F_\ell(h^{(\ell)})$. A residual block learns only what to **add**:

$$h^{(\ell+1)} = h^{(\ell)} + F_\ell(h^{(\ell)})$$

A Residual Block Learns a Correction

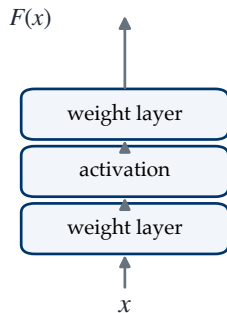
A plain block must learn the whole mapping, $h^{(\ell+1)} = F_\ell(h^{(\ell)})$. A residual block learns only what to **add**:

$$h^{(\ell+1)} = h^{(\ell)} + F_\ell(h^{(\ell)})$$

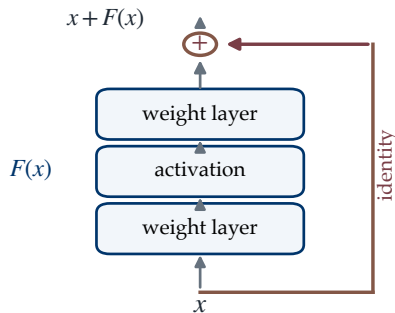
- ▶ If the useful transformation is near the identity, the branch has little left to learn;
- ▶ Setting $F_\ell = 0$ recovers the identity exactly, so extra depth **cannot** hurt the function class.

Plain Block against Residual Block

plain block: learn the whole map



residual block: learn a correction



Same two weight layers and activation in both. The residual block adds one wire: the input is carried around the block and **added** to its output, so the block only has to produce the correction.

The construction is from He et al. (2016).

The Skip Path Gives the Gradient a Direct Route

With $y = x + F(x)$ the block Jacobian is $\partial y / \partial x = I + J_F(x)$, so an incoming gradient $\bar{y}$ leaves as

$$\bar{x} = \underbrace{\bar{y}}_{\text{direct}} + \underbrace{J_F(x)^\top \bar{y}}_{\text{through the block}}$$

The Skip Path Gives the Gradient a Direct Route

With $y = x + F(x)$ the block Jacobian is $\partial y / \partial x = I + J_F(x)$, so an incoming gradient $\bar{y}$ leaves as

$$\bar{x} = \underbrace{\bar{y}}_{\text{direct}} + \underbrace{J_F(x)^T \bar{y}}_{\text{through the block}}$$

- ▶ The first term crosses **untouched**: nothing to shrink it;
- ▶ The second adds the learned correction;
- ▶ Stacked L deep, the direct terms give a path of factor **exactly one**.

The Skip Path Gives the Gradient a Direct Route

With $y = x + F(x)$ the block Jacobian is $\partial y / \partial x = I + J_F(x)$, so an incoming gradient $\bar{y}$ leaves as

$$\bar{x} = \underbrace{\bar{y}}_{\text{direct}} + \underbrace{J_F(x)^T \bar{y}}_{\text{through the block}}$$

- ▶ The first term crosses **untouched**: nothing to shrink it;
- ▶ The second adds the learned correction;
- ▶ Stacked L deep, the direct terms give a path of factor **exactly one**.

Trainability only: skips need matching shapes, and set neither the learning rate nor the amount of overfitting.

Two More Tricks, on the Optimizer Side

Initialization and skips keep the per-layer factor near one. Two further devices act on the [step](#) itself, when it is not.

Two More Tricks, on the Optimizer Side

Initialization and skips keep the per-layer factor near one. Two further devices act on the [step](#) itself, when it is not.

Gradient clipping. If the gradient is too long, shorten it but keep its direction:

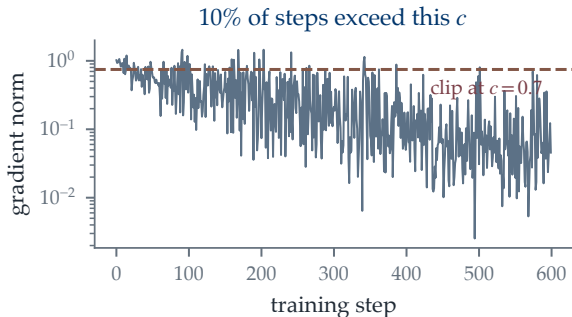
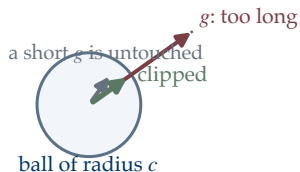
$$g \leftarrow g \cdot \min\left(1, \frac{c}{\|g\|_2}\right).$$

A single bad minibatch can then no longer throw the parameters across the landscape.

Standard in language-model training, where a rare long sequence can produce a gradient many times the usual size.

Clipping Caps the Length, Not the Direction

direction kept, length capped



Left: a gradient longer than c is pulled back onto the ball of radius c ; a short one is left alone.

Right: gradient norms during a real run of a three-layer network — mostly near 0.2, with spikes **eight times** larger that clipping removes.

Course computation on the handwritten-digits data; c set at the 90th percentile of the observed norms.

Learning-Rate Schedules

The step size need not be constant. A common schedule has two phases:

- ▶ **Warmup**: start small and increase over the first few thousand steps, while the parameters are still far from anything sensible and the curvature estimate is poor.
- ▶ **Decay**: reduce the rate as training proceeds — cosine or step decay — so late updates refine rather than jump.

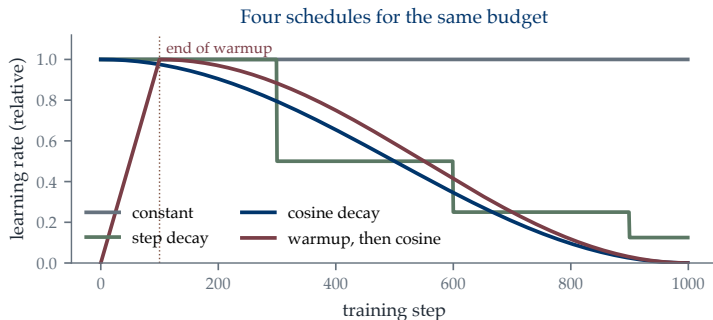
Learning-Rate Schedules

The step size need not be constant. A common schedule has two phases:

- ▶ **Warmup**: start small and increase over the first few thousand steps, while the parameters are still far from anything sensible and the curvature estimate is poor.
- ▶ **Decay**: reduce the rate as training proceeds — cosine or step decay — so late updates refine rather than jump.

None of the four devices changes the model class. They change whether the optimizer can **reach** a good member of it.

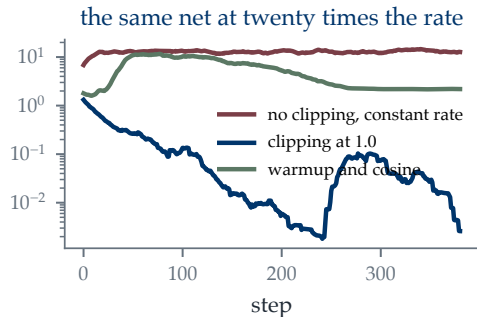
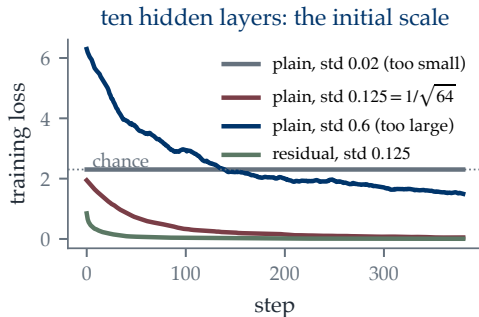
Four Schedules for the Same Budget



A constant rate, step decay, cosine decay, and a short warmup followed by cosine. Warmup keeps the first steps small while the parameters are still arbitrary; decay makes the last steps refine rather than jump.

Relative rates over a fixed budget of 1,000 steps.

The Tricks, Measured



Left: ten tanh layers on digits. At std 0.02 the signal dies and the loss sits at chance; at 0.6 the units saturate and it crawls to 1.5; at $1/\sqrt{64}$ it reaches 0.05, and the residual version 0.006. Right: at twenty times the rate the plain run **diverges to 12**, while clipping reaches 0.003.

SGD, batch 128, 400 steps, loss smoothed over 20; right panel on a log scale.

Outline

1. Why Deep Learning Now?
2. Depth and Learned Representations
3. Backpropagation and Automatic Differentiation
4. Stabilizing Deep Networks
5. Regularization and the Training Handoff

Dropout Randomly Removes Hidden Coordinates

For a hidden vector h and dropout probability p , draw $m_j \sim \text{Bernoulli}(1 - p)$ independently and use

$$\tilde{h}_j = \frac{m_j}{1 - p} h_j$$

Dropout Randomly Removes Hidden Coordinates

For a hidden vector h and dropout probability p , draw $m_j \sim \text{Bernoulli}(1 - p)$ independently and use

$$\tilde{h}_j = \frac{m_j}{1 - p} h_j$$

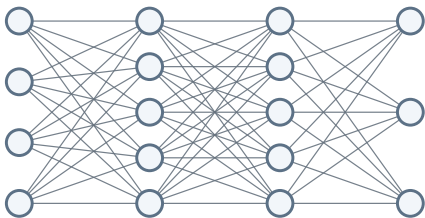
Dividing by $1 - p$ keeps the scale: $\mathbb{E}[\tilde{h}_j | h_j] = h_j$. Each minibatch therefore trains a **randomly thinned** network, and no single unit can be relied upon.

Source: Srivastava et al. (2014).

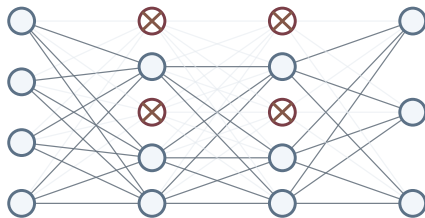
A Different Thinned Network Every Minibatch

Dropout trains a different thinned network on every minibatch

all units present



one dropout mask



Left: the full network. Right: one draw of the mask, with two hidden units in each layer switched off and every edge touching them idle. The next minibatch draws a **different** mask.

Dropout is from Srivastava et al. (2014).

Dropout Targets Overfitting, Not Gradient Instability

mode	transformation	purpose
training	$\tilde{h} = m \odot h / (1 - p)$	noisy regularized fitting
evaluation	$\tilde{h} = h$	deterministic prediction

Dropout discourages fragile co-adaptation of hidden coordinates and can reduce a train-validation gap. It may also make optimization noisier.

In PyTorch, `model.train()` activates dropout and `model.eval()` disables it. Forgetting the mode switch makes evaluation stochastic and changes the model being assessed.

Different Failure Modes Need Different Interventions

symptom	primary intervention	role
vanishing/exploding scale	Xavier or He	usable initial scale
degradation with depth	residual path	direct signal route
saturated activations	activation/normalization	responsive units
unstable updates	rate/clipping/optimizer	controlled motion
train-validation gap	dropout/decay/augmentation	regularization

These mechanisms solve different problems. Lecture 12 studies the update rule; Lecture 13 develops normalization in the concrete setting of image channels.

Summary: Backpropagation and Initialization

1. For any intermediate v , $\bar{v} = \nabla_v L$ has the same shape as v .
2. Backpropagation combines an incoming gradient with each local derivative.
3. Initialization controls the starting scale; residual paths provide a direct route through depth.

$$\overline{W}^{(\ell)} = \bar{z}^{(\ell)} h^{(\ell-1)\top}, \quad h^{(\ell+1)} = h^{(\ell)} + F_{\ell}(h^{(\ell)}).$$

$$\text{Var}(W_{jk}) \approx \frac{2}{d_{\text{in}} + d_{\text{out}}} \text{ (Xavier)}, \quad \text{Var}(W_{jk}) \approx \frac{2}{d_{\text{in}}} \text{ (He/ReLU)}.$$

Summary: Regularization Is a Separate Question

1. Dropout masks hidden coordinates during training and rescales survivors so the conditional expectation is unchanged.
2. Dropout is disabled for deterministic evaluation.
3. It targets overfitting; residual connections and initialization target trainability.

$$\tilde{h}_j = \frac{m_j}{1-p} h_j, \quad m_j \sim \text{Bernoulli}(1-p).$$

Next Lecture

Backpropagation supplies a gradient, while initialization and residual paths keep the network trainable. A practical run still depends on which examples define each update, what state the optimizer stores, and how its rate changes.

Lecture 12 — Optimization for Deep Learning connects shuffled data loaders, minibatches, epochs, schedules, and adaptive optimizers in one complete training procedure.

Reading for this lecture: ISLP Chapter 10; D2L Chapter 5, Multilayer Perceptrons; DL Chapter 6.